

基于动作偏好优化的人类辅助机器人 policy 精炼

Wenke Xia^{1,2,3,4,*}, Yichu Yang², Hongtao Wu², Xiao Ma², Tao Kong², Di Hu^{1,3,4,†}

¹ 中国人民大学高瓴人工智能学院, 北京

²ByteDance Seed

³ 教育部下一代智能搜索与推荐工程研究中心

⁴ 北京市大模型与智能治理重点实验室

* 本工作于 ByteDance Seed 完成

† 通讯作者

2026 年 8 月 17 日

摘要

构建一个可靠且能够持续迭代优化的机器人系统, 是推动其在真实世界中落地应用的关键。尽管 Vision-Language-Action (VLA) 模型已被广泛视为支撑此类机器人部署的基础模型, 但其对离线 expert demonstration 的依赖, 严重限制了模型在部署后的持续优化能力。为缓解这一问题, 我们提出 Action Preference Optimization (APO): 一种利用人类辅助的偏好对齐信号, 并通过与环境交互来优化 VLA 模型的方法。

该方法首先构建了一个人机协作框架: 在人类介入的帮助下, 实现可靠的失败纠正, 并收集交互轨迹。然而, 要将这些交互轨迹直接用于偏好优化并不容易, 因为机器人动作具有不可逆性, 同时还存在 token 分布失配的问题。针对这些挑战, APO 提出了一种自适应重加权算法, 利用从交互过程中提取的二值 desirability 信号, 使 VLA 模型能够有效抑制容易导致失败的动作, 并增强其对纠正性动作的适应能力。

最终, APO 为 VLA 模型赋予了从失败中学习这一关键能力, 从而为其在动态环境中的迭代优化与可靠部署奠定基础。仿真与真实场景中的实验结果表明, 我们提出的人类辅助框架在多种 manipulation 任务上都展现出更强的泛化性与鲁棒性。我们相信, 这项工作能够为基于人机协作的 VLA 模型高效、稳定优化提供有价值的启发。

日期: 2025 年 10 月 31 日

联系人: Di Hu, dihu@ruc.edu.com

项目主页: https://gewu-lab.github.io/action_preference_optimization/

代码: <https://github.com/GeWu-Lab/Action-Preference-Optimization>

1 Introduction

在真实场景中, 持续提升能力对于构建鲁棒的机器人操纵系统至关重要 [10, 28, 45]。得益于可泛化推理与可扩展学习能力, Vision-Language-Action (VLA) 模型 [3, 4, 8, 18, 21, 43, 47] 已被广泛视为此类机器人部署系统的基础模型。然而, 这类模型当前主流的训练范式高度依赖大规模专家

demonstration 数据集。这一特点严重限制了它们在部署后的进一步优化，因为它们缺乏从失败中持续学习、以及适应真实环境中新场景的内在能力。

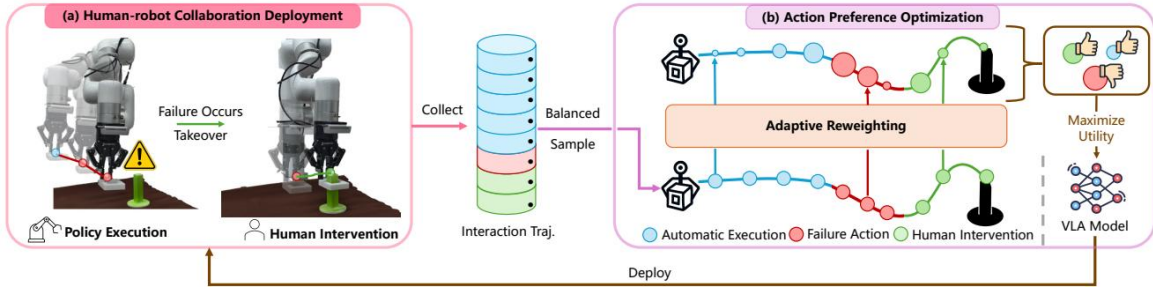


图 1: 我们的方法包含两个关键组成部分: (a) 人机协作部署框架, 用于实现可靠部署, 并在人工干预下采集交互轨迹; (b) 带有自适应重加权的 action preference optimization 过程, 用于让 VLA 模型从次优交互轨迹中学习。图中每个圆的大小表示其在训练过程中的权重。

为增强机器人系统的持续学习能力, 研究者提出了交互式模仿学习框架 [7, 17], 通过迭代式的人在回路 (human-in-the-loop) 纠错反馈来修正易出错轨迹。其中, behavior cloning [15, 22, 35] 被广泛用于以监督学习方式, 利用人工修正后的干预数据对基础 policy 模型进行微调 (fine-tuning)。相比之下, 近期方法 [20, 26] 试图基于次优人工干预轨迹, 设计有效的 on-policy 强化学习算法。然而, behavior cloning 无法充分利用失败轨迹中的信息, 而这些失败轨迹恰恰是学习鲁棒 policy 的重要信号。与此同时, 强化学习方法在训练大规模 VLA 模型时又面临显著的可扩展性限制, 原因在于其训练过程本身不稳定, 且难以构建具有泛化能力的 value function。到目前为止, 如何在次优人工干预范式下有效适配 VLA 模型以服务下游操纵任务, 仍缺乏充分研究。

为弥补这一空白, 我们提出 Action Preference Optimization (APO)。这一新范式突破了 behavior cloning 与强化学习各自的局限。具体而言, 该方法通过学习交互过程中体现出的动作级偏好 (action-level preferences), 在保持大规模 VLA 模型所需优化稳定性的同时, 充分挖掘失败轨迹中的有效信息。

如图 1(a) 所示, 我们的方法建立在人机协作框架之上。该框架既能保证部署过程的可靠性, 又能同时生成用于 policy 优化的数据。当机器人遇到困难情形时, 实时人工干预不仅能够确保任务成功完成, 还会提供纠正轨迹。这些轨迹随后被构造成 preference pairs, 用于进一步优化 policy。此外, 针对所采集数据中不同动作类型分布不均衡的问题, 我们采用了一种均衡采样方法, 以确保各类交互数据都能在后续 VLA preference optimization 中得到成比例的代表。

2 1 引言 (第二部分)

然而, 将这类偏好数据直接用于微调自回归 VLA 模型, 会面临两个彼此关联且都十分关键的挑战: (1) **不可逆交互**: LLM 通常依赖成对的偏好数据, 但物理交互具有不可逆性, 因此几乎无法在完全相同的条件下采集到理想的正负动作样本对。(2) **token 概率失配**: 自回归 VLA 模型会将连续动作离散化为 token, 这会在 token 概率与真实动作损失之间引入根本性失配, 从而使偏好对齐变得更加复杂。

针对上述紧迫问题, 我们首先引入 Kahneman 和 Tversky 的前景理论 (prospect theory) [11, 40], 构建了一种新的偏好对齐目标。该目标从交互过程中提取出的二值可取性信号中学习。相比依赖严格成对偏好数据的做法, 这一目标显著放宽了对偏好样本形式的要求, 因此更适合从具有不可

逆性的机器人交互轨迹中学习。

进一步地，我们提出了一种自适应重加权方法，利用解码后的连续动作来有效引导离散动作 token 空间中的偏好优化。该方法通过对每个样本的训练权重进行动态调制，缓解了动作 token 概率失配带来的问题，并将梯度优化集中到更容易失败的交互动作上。借助这种有针对性的权重细化，我们成功将偏好对齐优化应用到 VLA 模型中，从而提升了模型从次优操纵纠正轨迹中学习时的性能。

为了系统评估所提系统的有效性，我们在 RoboMimic [29] 仿真环境中开展了全面实验。实验结果表明，APO 能够在分布内场景中实现快速自适应，同时在多种未见扰动下保持稳健性能。此外，终身学习实验进一步表明，该框架能够在人的干预下进行迭代改进。为验证该框架在实际场景中的可行性，我们还在多种扰动条件下开展了真实机器人细粒度插入任务实验，结果展示了该方法在真实机器人操纵场景中的鲁棒性与应用潜力。

3 2 Related Works

3.1 Vision-Language-Action Models

实现具有良好泛化能力的机器人操纵，仍然是机器人学中的一项重要挑战。受到基础模型 (foundation models) 近期进展 [19, 32, 39, 42, 44, 48] 的推动，一些工作 [6, 12, 41] 尝试构建大规模真实世界机器人数据集，以促进具备泛化能力的 Vision-Language-Action (VLA) 模型的发展。基于这些数据集，近期研究 [5, 18, 33] 将机器人动作预测表述为 VLM 框架下的 next token prediction 问题。与此不同，另一些研究 [24, 38] 探索将 diffusion-based 方法用于建模多模态动作分布，从而提升操纵任务中的鲁棒性。

上述工作主要聚焦于基于专家 demonstration 的行为克隆 (behavior cloning)。与之相比，Grape [49] 提出了一种轨迹级偏好对齐方法，通过同时引入成功与失败试验来提升泛化能力。然而，该方法要求在相同条件下获得成对轨迹，这一前提在真实世界场景中往往难以满足。本文提出 action preference optimization 方法，通过融合 human-in-the-loop 的干预偏好数据，对 VLA 模型进行持续优化。

3.2 大语言模型的偏好对齐

当前主流方法 [9, 30, 31, 50] 通常采用两阶段范式实现基于人类反馈的强化学习 (Reinforcement Learning from Human Feedback, RLHF)：首先训练一个奖励估计模型，然后利用强化学习方法 [13, 25, 36] 优化 LLM，使其最大化该估计奖励。然而，这一范式在实际应用中往往存在训练缓慢且不稳定的问题。DPO [34] 提出了一种单步替代方案：将 RLHF 的优化目标重新参数化为闭式损失函数，从而直接最大化偏好输出与非偏好输出之间的对数似然间隔。在此基础上，KTO [11] 进一步引入了 human-aware loss，可基于“某个输出是否符合期望”这一二元信号进行学习，从而无需复杂的偏好标注。

在本文中，我们将偏好对齐优化方法扩展到 Vision-Language-Action 模型。具体而言，我们提出了一种自适应重加权策略，用于缓解将偏好学习方法从 LLM 迁移到 VLA 模型时面临的两类关键挑战：不可逆交互问题，以及 token 概率失配问题。

3.3 2.3 人机交互学习

交互式模仿学习 (interactive imitation learning) [1, 7] 被提出用于通过人类反馈细化机器人的动作。早期研究 [17, 22, 35] 往往需要持续的人类监督，以便在机器人执行动作时进行干预；而较新

的工作 [14, 23, 46] 则引入了用于自动失败检测与实时监控的动态模型。相比之下, RLIF [26] 将人类干预信号用作 off-policy RL 的奖励; HIL-SERL [27] 提出了一种 human-in-the-loop、基于视觉的 RL 系统, 面向灵巧操纵任务进行了专门设计。然而, 这类 RL 方法在大规模 VLA 模型训练中会遇到困难, 主要原因在于梯度优化不稳定。为此, 本文提出了 action preference optimization 方法, 以利用交互过程中采集到的动作级偏好信息, 实现 policy 的稳定优化。

4.3 方法

本节介绍 Action Preference Optimization (APO)。该方法旨在促进 Vision-Language-Action (VLA) 模型的持续迭代改进。正如算法 1 所示, APO 通过在人机协作部署过程中收集的人类偏好信号, 使模型逐步与人类偏好对齐。

算法 1: Action Preference Optimization**记号:** $\mathcal{D}_e$: expert demonstrations, $\mathcal{D}_h$: 交互数据集, π : 交互 policy**Warm-start 阶段**1: 收集 $\mathcal{D}_e\{\tau_1^e, \dots, \tau_N^e\}$ 2: 初始化 BC policy π_θ^0 3: $\theta^* \leftarrow \arg \max_\theta \mathbb{E}_{(o,a) \sim \mathcal{D}^e} -\log \pi_\theta^0(a|o)$ 4: $\mathcal{D}_h^0 \mathcal{D}_e$ **部署-优化循环**5: for $i \leftarrow 0$ to X do6: π_{ref}, π_θ^i 7: $\mathcal{D}_h^{i+1} \leftarrow \text{DEPLOYMENT}(\pi_\theta^i, \mathcal{D}_h^i)$ 8: $\pi_\theta^{i*+1} \leftarrow \text{OPTIMIZATION}(\pi_\theta^i, \pi_{ref}^*, \mathcal{D}_h^{i+1})$

9: end for

函数 $\text{DEPLOYMENT}(\pi_\theta, \mathcal{D}_h)$

10: for n interaction rollouts do

11: while task does not succeed do

12: if human intervenes then

13: $a_t \leftarrow human, c_t \leftarrow 2$ 14: $c_{t-K:t-1} \leftarrow 0$

15: else

16: $a_t \pi_\theta(o_t), c_t 1$

17: end if

18: $\tau_i^h \tau_i^h \cup (o_t, a_t, c_t)$

19: end while

20: $\mathcal{D}_h \mathcal{D}_h \cup \tau_i^h$

21: end for

22: return $\mathcal{D}_h$ **函数** $\text{OPTIMIZATION}(\pi_\theta, \pi_{ref}, \mathcal{D}_h)$

23: for n gradient steps do

24: Balanced Sample $(o_i, a_i, c_i)_{i=1}^{i=B} \sim \mathcal{D}_h$ 25: $l_i \leftarrow |\pi_\theta(o_i) - a_i|_1$ 26: $w_i \leftarrow \frac{l_i}{\sum_{i=1}^B l_i}$ 27: if $c_i \neq 0$ then28: $\lambda_{D_i} = 1 - e^{-\beta_D * w_i}$

29: else

30: $\lambda_{U_i} = e^{-\beta_U * w_i}$

31: end if

32: $\theta^* = \arg \max_\theta \mathbb{E}[-v(o, a, \pi_\theta, \pi_{ref}, \lambda_D, \lambda_U)]$

33: end for

34: return π_{θ^*}

35: end function

4.1 人机协作部署

为确保系统能够可靠部署，并高效采集交互轨迹，我们的方法建立在人机协作框架之上，以支持实时人工干预和交互数据获取。

首先，我们收集专家演示数据集 $\mathcal{D}_e = \{\tau_e^i\}_{i=1}^N$ 。其中，每条轨迹 τ_e^i 由带有专家标注的观测-动作对构成： $\tau_e^i = \{(o_t^i, a_t^i, c_t^i)\}_{t=1}^{\bar{T}}$ ，其中， $c_t^i = 1$ 表示动作 a_t^i 由人类专家执行。我们采用行为克隆对预训练 VLA 模型在这些专家演示上进行微调 (fine-tuning)，得到初始 base policy π_θ^0 。随后，将该 policy 部署到真实环境中，用于采集交互轨迹。

在 policy 执行过程中，人类操作员持续监控其执行情况；当 policy 遇到困难场景时，操作员会进行干预。通过这一过程，我们可以收集一组交互轨迹 $\mathcal{D}_h = \{\tau_h^i\}_{i=1}^M$ 。其中， $c_t^i = 2$ 表示该动作经过人工干预修正，而 $c_t^i = 1$ 表示该动作由 policy 执行。

进一步地，我们对交互轨迹重新标注：将每次人工干预发生前 K 步内采取的动作归类为不理想动作，并标注为 $c_t^i = 0$ 。对于每条轨迹，我们将连续动作 a 离散化为离散 action token $\hat{a}$ 。最后，我们将专家演示数据 $\mathcal{D}_e$ 与交互数据集 $\mathcal{D}_h$ 合并，用于后续的机器人动作偏好优化。

4.2 3.2 Action Preference Optimization (第一部分)

为尽可能利用次优交互轨迹，并保证 VLA 模型在微调 (fine-tuning) 过程中的稳定性，我们采用偏好对齐优化 (preference alignment optimization) 方法，引导模型从纠正信号中学习，并规避失败动作。

尽管以人类反馈强化学习 (Reinforcement Learning with Human Feedback, RLHF) 为代表的既有方法 [2, 34] 已被证明对 LLM 微调有效，但在机器人操纵场景下，对 VLA 模型进行偏好优化还面临额外挑战：

- 机器人操纵过程通常不可逆，因此很难在相同观测条件下获得有意义的正负配对动作。
- 自回归 VLA 将连续机器人动作映射为离散 token，这会导致 token 概率与连续动作误差之间出现失配，从而增加基于 action token 预测进行偏好优化的难度。

为解决上述问题，我们采用 Kahneman 与 Tversky 的前景理论 (prospect theory) [40]，基于二值 desirability 信号进行偏好对齐优化，并提出一种自适应重加权方法，以弥合离散 token 预测与连续动作回归之间的差距。首先，我们按照标准做法 [31, 34, 37]，对模型 π_θ 的奖励函数 r_θ 进行估计：

$$r_\theta(o, \hat{a}) = \log \frac{\pi_\theta(\hat{a}|o)}{\pi_{\text{ref}}(\hat{a}|o)}, \quad (1)$$

其中， $\hat{a}$ 表示离散 action token；参考模型 π_{ref} 则是在算法 1 所示的每一轮 deployment-optimization 循环开始时，对应的基础模型 π_θ^i 。参考 [11, 40]，我们将效用函数 v 定义如下，用于估计机器人数据上的相对收益：

$$v(o, \hat{a}) = [\text{formula}] \quad (2)$$

其中， λ_D 和 λ_U 用于重要性采样， σ 为 sigmoid 函数。为避免模型 π_θ 相对参考模型 π_{ref} 偏离过大，我们引入惩罚项 z_0 。该项被定义为 π_θ 与 π_{ref} 之间的 KL-Divergence，即 $z_0 = KL(\pi_\theta || \pi_{\text{ref}})$ 。将 z_0 纳入优化过程后，模型在利用偏好对数据学习的同时，也能保留先前模型中已经获得的知识。基于 desirability 信号，我们采用如下损失函数 L 对模型 π_θ 进行偏好优化：

$$L(\pi_\theta, \pi_{ref}) = \mathbb{E}_{x,y \sim D^h} [-v(x, y)]. \quad (3)$$

通过最小化该损失函数，我们希望模型 π_θ 相较于参考模型 π_{ref} ，能够对 desirable 样本对赋予更高奖励，同时避免预测 undesirable 动作。

然而，若直接将 LLM 中的偏好对齐优化方法应用到自回归 VLA 模型上，会遇到明显问题，核心原因在于二者的 token 定义并不相同。对于语言模型，word token 对应的是明确的子词单元；而 action token 则需要通过一个不可微映射，才能对应到连续的真实动作。这会造成 token 分类概率与连续机器人动作对应的回归损失之间存在偏差。

为弥合自回归 VLA 模型中 token 分类与连续动作回归之间的差距，我们进一步提出一种自适应重加权方法。其基本思想是：优先关注那些回归误差较大的样本。具体而言，我们先为每个样本估计连续动作的 L1 损失 l ，再在 batch 层面进行归一化，形式如下：

$$w_i = \frac{l_i}{\sum_{i=1}^{i=B} l_i}. \quad (4)$$

4.3 3.2 Action Preference Optimization（第二部分）

归一化加权方案的工作方式如下：1) 对于 desirable data，提高动作预测误差较大样本的权重；2) 对于 undesirable data，提高那些动作与失败动作更接近样本的权重。通过利用归一化权重，自适应地调整式 7 中 λ_D 和 λ_U 的取值，我们能够在训练过程中对每个样本的相对影响进行更细粒度的控制：

$$\lambda_D = 1 - e^{-\beta_D * w}, \quad (5)$$

$$\lambda_U = e^{-\beta_U * w}. \quad (6)$$

通过引入基于样本级权重细化的偏好对齐优化，我们在从次优操纵纠正轨迹中学习时，提升了 VLA 模型的性能与优化稳定性。

总而言之，我们提出了算法 1 所示的 action preference optimization 方法。该方法利用人机协作部署来实现可靠的任务执行与交互轨迹采集，而 action preference optimization 过程则通过自适应重加权，为自回归 VLA 的优化提供稳定性。借助人机协作部署与 action preference optimization 的迭代进行，自回归 VLA 模型能够从与环境的持续交互中实现持续改进。

5 4 实验

为全面评估我们提出的动作偏好优化方法在高效下游适应中的表现，我们设计了一系列实验，以验证以下问题：

- APO 在促进模型适应分布内场景方面有多有效？见第 4.2 节。
- 面对多种扰动时，APO 在新场景中是否仍能保持有效的学习性能？见第 4.3 节。
- APO 在部署过程中是否具备迭代改进的能力？见第 4.4 节。
- APO 对不同 VLA 模型的泛化能力如何？见第 4.5 节。

- APO 是否能够应用于细粒度的真实世界场景？见第 4.6 节。
- 动作级偏好优化方法在多大程度上使 APO 能够学习动作修正？见第 4.7 节。

5.1 4.1 实验设置

实现细节。 本文以 OpenVLA [41] 作为基础模型, 并针对目标操纵任务对其进行微调 (fine-tuning)。在参数高效调优方面, 我们采用 LoRA [16], 设定 rank $r = 32$, 在 8 张 NVIDIA A100 GPU 上以 batch size 为 16 进行训练。进一步地, 我们将基础模型部署到交互环境中, 使其与环境进行交互; 当执行过程中出现失败时, 人类操作员通过 SpaceMouse 设备进行实时纠错干预, 以修正失败行为。我们设定 $K = 10$, 用于自动识别并标注不期望行为。

图 2 展示了本文的人类辅助交互轨迹。根据人类纠正发生的时间点, 整段轨迹被划分为机器人自动执行、失败动作以及人类干预类型三个部分。基于任务执行过程中采集到的交互轨迹, 我们进一步使用所提出的动作偏好优化方法对基础模型 π_{ref} 进行微调, 训练时学习率设为 $5e-5$, 在 4 张 NVIDIA A100 GPU 上以 batch size 为 8 进行训练。为保证偏好对齐训练的稳定性, 我们采用均衡采样策略, 使每个 batch 中包含 50% 的 expert actions、25% 的人类干预动作以及 25% 的失败动作。



图 3: 本文人类辅助交互轨迹的示意图。

仿真环境细节。 为进行全面评估, 我们在 RoboMimic [29] 仿真环境中的细粒度操纵任务上验证这些方法, 例如 “make coffee” 和 “toy assembly”。在 RoboMimic 环境中, 我们基于预训练的 OpenVLA 模型, 针对 4 个长时程操纵任务进行微调, 每个任务使用 300 条 expert demonstrations。为利用人类偏好优化 policy, 我们在 RoboMimic 任务中针对每个任务、在不同随机种子下采集 50 条轨迹。评估时, 我们对每个任务在 3 个未见过的随机种子下各进行 50 次试验, 并报告平均成功率 (success rate)。

5.2 4.2 对比实验

我们将 APO 与其他方法进行对比, 以评估其在 VLA 模型微调 (fine-tuning) 中的有效性。为保证公平性, 我们以 OpenVLA [?] 作为 manipulation 任务的 base model, 并在该 base model 上分别采用其他对比方法进行改进。

- **Dagger [?]**: 将专家 demonstration 与交互轨迹混合, 并采用 behavior cloning 目标对 base model 进行微调。
- **Sirius [?]**: 对样本进行重加权, 使人工干预数据具有更高优先级, 并基于加权的 behavior cloning loss 对 base model 进行微调。
- **DPO [?]**: 针对交互轨迹, 通过对 base model 预测动作加入 Gaussian noise 扰动来构造配对的负样本, 再利用这些配对数据采用 DPO 方法对 base model 进行微调。
- **TPO [?]**: 根据干预发生的时刻, 在交互轨迹中选择正负样本, 然后采用基于整条轨迹的 preference optimization 对模型进行微调。

- **KTO [?]**: 同样根据干预时刻在交互轨迹中选择正负样本。随后采样正负轨迹, 并在约束 $z_0 = KL(\pi_\theta || \pi_{ref})$ 下, 使用 KTO 对 base model 进行优化。

如表 1 所示, 我们首先比较了基于 behavior cloning 目标的方法。结果表明, 在使用交互数据进行微调后, 这些方法均未能超过 base model。这说明现有的 behavior cloning 方法在大规模 VLA 模型场景下, 难以实现高效自适应。其关键挑战在于专家轨迹与交互轨迹之间存在分布偏移 (distribution shift)。在标准 behavior cloning 目标下, 如果缺乏保留 base model 原有知识的机制, 这种分布偏移会使大规模 VLA 模型更难有效拟合由专家数据与交互数据混合形成的复杂多模态分布。

我们进一步给出了基于 preference optimization 方法的结果。由于引入了与参考模型 π_{ref} 相关的正则约束, 这类方法能够在利用交互轨迹获得性能提升的同时, 保留参考模型中的有用知识。

在所有基于 preference learning 的对比方法中, DPO 的表现最弱。其原因在于, DPO 的优化仅依赖合成得到的成对失败动作, 而没有接触真实世界中的错误; 但对于通过交互教会机器人规避错误而言, 真实错误样本是至关重要的。另一方面, TPO 在多个任务上未能带来稳定的性能增益, 而 APO 相对于 base model 则获得了稳定提升。TPO 使用负样本来约束模型的 preference alignment 优化, 但其随机采样机制会引入不稳定性。相比之下, APO 利用 KL divergence 估计更新后模型与参考模型之间的平均间隔, 这不仅使学习过程更加稳定, 也更有利于保留先验知识。与 KTO 相比, APO leverages the

表 1: RoboMimic Simulation 上 4 个 manipulation 任务的对比实验结果。结果表明, 相比其他 behavior cloning 方法和 preference optimization 方法, 我们的 adaptive reweighting preference optimization 方法能够带来更稳定的性能提升。

Methods	Coffee_D0	StackThree_D0	ThreePieceAsseby_D0	Square_D0	Mean
Base policy	44%	46%	44%	28%	40.5%
Dagger [?]	42%	50%	36%	28%	39.0%
Sirius [?]	34%	52%	34%	38%	39.5%
DPO [?]	52%	46%	28%	22%	37.0%
TPO [?]	54%	54%	40%	18%	41.5%
KTO [?]	48%	52%	46%	32%	43.5%
APO	60%	54%	46%	32%	48.0%

5.3 4.2 对比实验（第 2 部分）

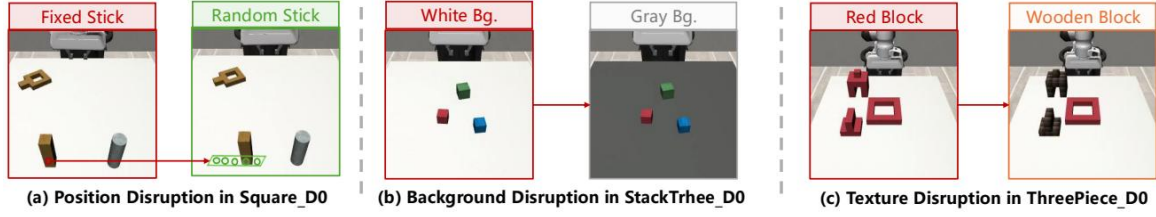


图 4: 扰动场景示意图。在位置扰动设置中，我们将 Square_D0 任务中 stick 的位置由固定点改为从图 (a) 所示矩形区域内随机采样的位置；在背景扰动设置中，我们将 StackThree_D0 任务的背景替换为图 (b) 所示的灰色背景；在纹理扰动设置中，我们将红色方块替换为木质方块。

表 2: 扰动场景上的实验结果。

Methods	Pos Dis.	Bg Dis.	Tex Dis.	Mean
Base policy	12%	42%	10%	21.3%
Dagger	18%	46%	4%	22.7%
Sirius	12%	42%	2%	18.7%
DPO	14%	26%	2%	14.0%
TPO	18%	32%	8%	19.3%
KTO	20%	46%	6%	24.0%
APO	26%	46%	12%	28.0%

表 3: 原始任务上的实验结果。

Methods	Square	StackThree	ThreePiece	Mean
Base policy	28%	46%	44%	39.3%
Dagger	16%	46%	30%	30.7%
Sirius	18%	48%	18%	28%
DPO	20%	50%	30%	33.3%
TPO	30%	36%	40%	35.3%
KTO	30%	46%	42%	39.3%
APO	34%	62%	40%	45.3%

APO 采用 adaptive reweighting 方法，从而能够更精确地控制正样本与负样本的重要性权重，并带来更显著的性能提升。

5.4 4.3 对新任务的泛化能力

本节在三种新的扰动场景下评估 APO 的泛化能力, 如图 3 所示。(1) **位置扰动 (Position Disruption)**: 在 Square_D0 任务中, 将原本固定的初始 stick 位置替换为操作区域内有界随机放置; (2) **背景扰动 (Background Disruption)**: 在 StackThree_D0 任务中, 将默认的白色背景替换为灰色背景; (3) **纹理扰动 (Texture Disruption)**: 在 ThreePiece_D0 任务中, 将原始红色方块替换为木纹外观。上述实验系统考察了模型对空间位置、背景以及视觉纹理变化的鲁棒性。为在新的扰动场景上微调基础模型, 我们在扰动场景下采集了 20 条交互轨迹, 并与原始任务中的 20 条 expert demonstration 合并, 用于后续微调。

我们的目标是构建一种支持持续改进的 action preference optimization 方法, 使模型在新的扰动场景中获得性能提升的同时, 在微调过程中保留原始任务能力。因此, 我们同时评估了微调后模型在扰动场景和原始场景中的表现。

如表 2 所示, 基础 policy 在扰动场景下存在一定程度的性能下降。不过, 背景扰动带来的退化相对较小, 而物体纹理与位置的扰动则会显著影响性能。无论是 behavior cloning 方法还是 preference optimization 方法, 在新的扰动场景中都难以获得明显的性能提升。相比之下, APO 能够通过自适应重加权 (adaptive reweighting) 有效适应新的扰动场景。

表 3 展示了模型利用扰动数据微调后, 在原始任务上的性能。结果表明, behavior cloning 方法存在严重的灾难性遗忘 (catastrophic forgetting), 从而导致性能大幅下降。相较之下, preference optimization 方法在参考模型约束下能够缓解性能退化。

此外, APO 通过自适应重加权机制, 有效融合了 expert demonstrations 与交互轨迹中的知识。该机制不仅有助于从多样化数据源中学习, 也进一步提升了模型在原始任务上的表现。

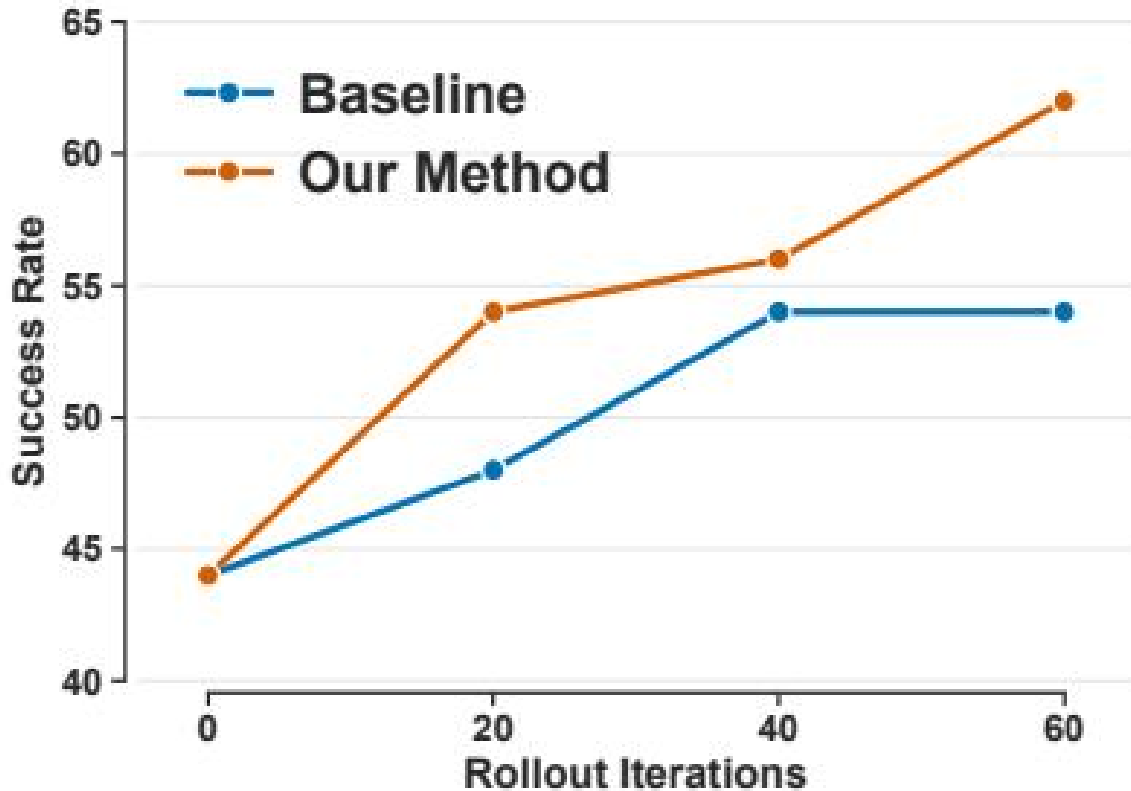


图 5: (a) Coffee_D0 任务成功率 (success rate)

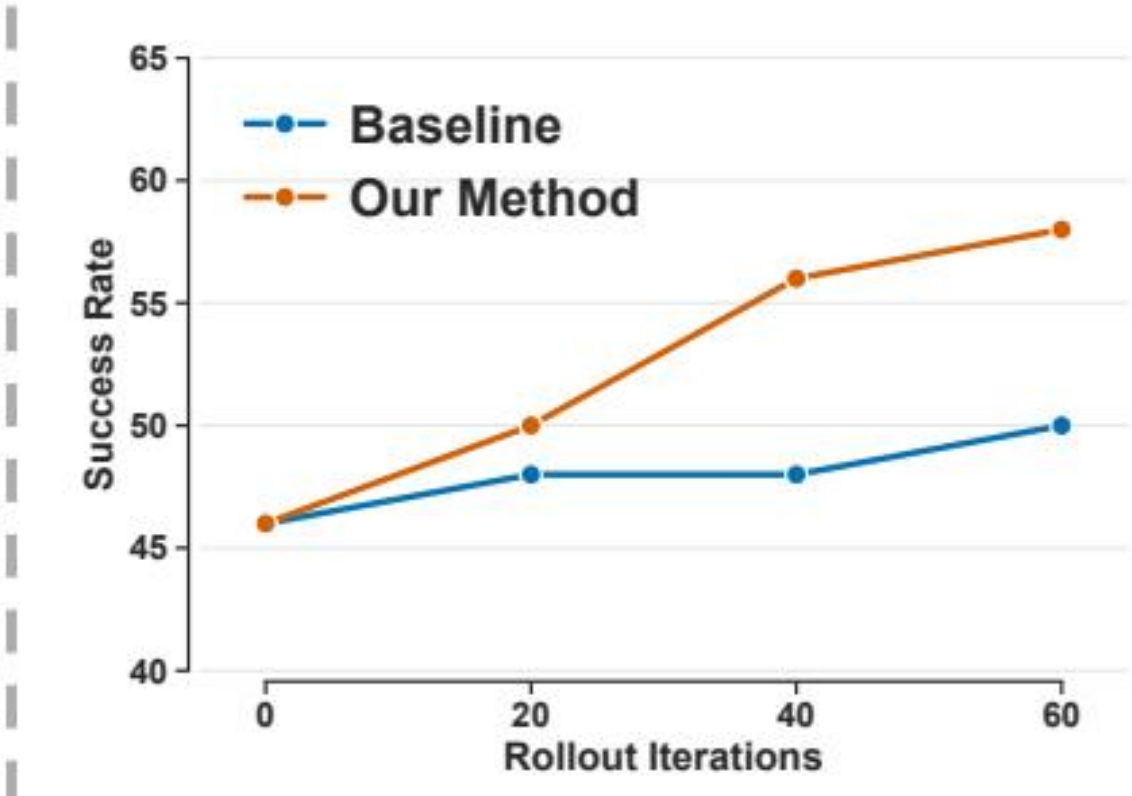


图 6: (b) StackThree_D0 任务成功率 (success rate)

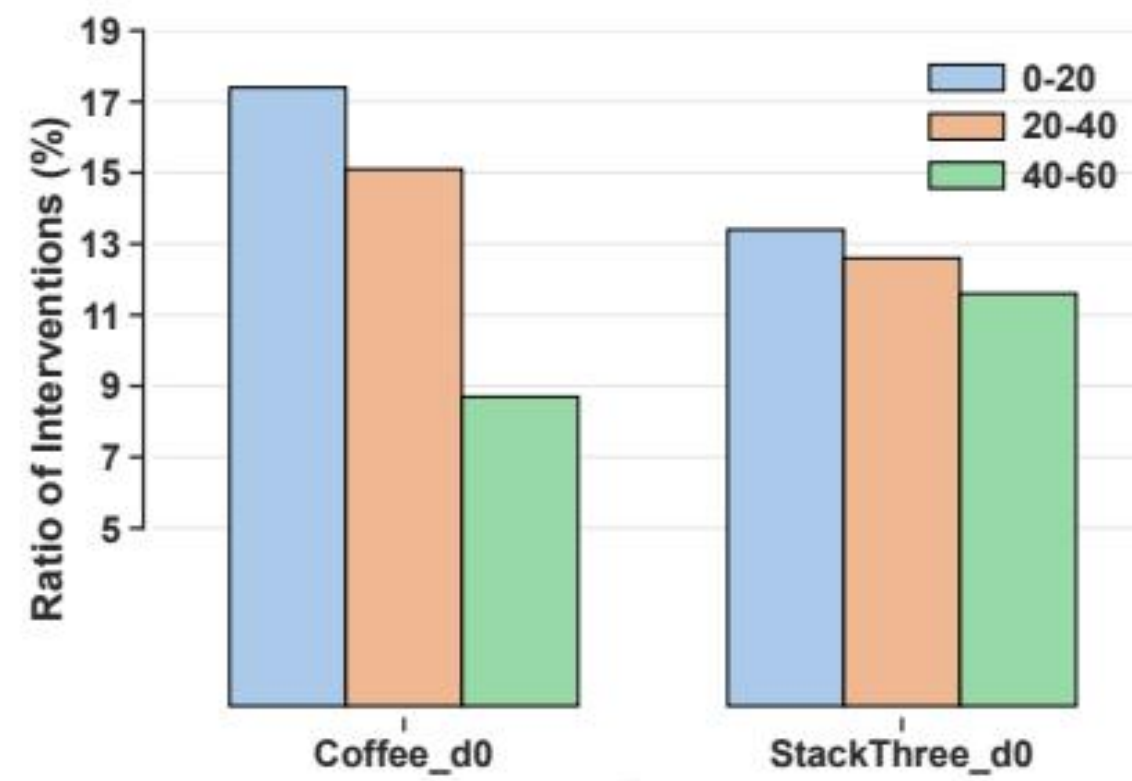


图 7: (c) 人工干预频率。图 4: APO 方法的终身学习结果。

表 4: 表 4: π 0-FAST 模型上的实验结果。

Methods	Coffee_D0	StackThree_D0	Insert Square
Base policy	68%	64%	85%
Dagger	64%	66%	85%
TPO	48%	52%	90%
APO	76%	74%	95%

表 5: 表 5: 真实世界实验结果。

Methods	In Dis.	Pos Dis.	Bg Dis.	Tex Dis.
Base policy	65%	25%	10%	25%
Dagger	65%	10%	10%	25%
TPO	75%	40%	20%	45%
APO	85%	55%	30%	55%

5.5 4.4 持续学习的性能

为研究 APO 是否能够通过与环境交互实现迭代式性能提升, 我们部署 APO 与环境进行交互, 并且每完成 20 次 interaction rollout 就更新一次模型。作为对比, 我们给出 baseline 结果: 该 baseline 为行为克隆 (behavior cloning) policy, 且使用与 APO 基线设置中相同数量的专家 demonstration 进行训练。对于每个更新后的模型, 我们进行 50 次试验, 并报告成功率 (success rate)。

如图 4(a-b) 所示, 相比 baseline, APO 表现更优, 这表明它能够有效利用次优的人类干预轨迹, 对模型进行迭代改进。当基础 policy 随着专家 demonstration 数量增加而逐渐出现性能提升趋缓时, APO 仍然能够继续从交互轨迹中获得性能增益。此外, 这一性能提升趋势还伴随着所需人类干预比例的相应下降, 如图 4(c) 所示。

5.6 4.5 对多种 VLA 模型的泛化能力

为验证 APO 能否适配不同的 VLA 模型, 我们将 APO 用于微调 (fine-tuning) π 0-FAST [33] 模型。 π 0-FAST 采用离散余弦变换编码 (discrete cosine transform encoding), 将 action chunking 编码为离散 token, 以用于 VLA 训练。为将该模型适配到下游任务中, 我们针对每个任务重新生成了 action tokenizer, 并将 action chunking 步数设为 5。

如表 6 所示, 基础模型相较于 OpenVLA 模型能够取得更高的成功率 (success rate), 这得益于其能够预测适用于机器人操纵的 action chunking。进一步地, 我们将 APO 与 behavior cloning 方法以及 preference optimization 方法进行比较。结果表明, 在 π 0-FAST 的微调中, APO 也能带来稳定的性能提升。

这些结果说明, APO 可以应用于多种 VLA 模型的微调, 并获得一致的性能增益。

5.7 4.6 真实世界实验

在本工作中，我们开展了一项具有挑战性的细粒度机器人操纵任务 “Insert the square into the stick”，如图 5(a) 所示。该任务要求机器人先抓取 square，再将其精确插入 stick 中。为采集专家 demonstration，我们使用 spacemouse 设备，以 20 Hz 的动作频率采集了 100 条高质量轨迹。随后，我们以收集到的 demonstration 为基础，对 OpenVLA 模型进行微调 (fine-tuning)，得到 base model。进一步地，我们将该 base model 部署到环境中进行交互，并提出实时 human-in-the-loop intervention 机制，额外采集了 20 条交互轨迹，用于后续的动作偏好优化。所有方法均在相同的实验设置下进行评估，并报告 20 次试验的平均成功率 (success rate)。为了更全面地评估 APO 在真实场景中的表现，我们还在 “hang cup on the rack” 和 “put lemon on the plate” 两个任务上进行了测试。

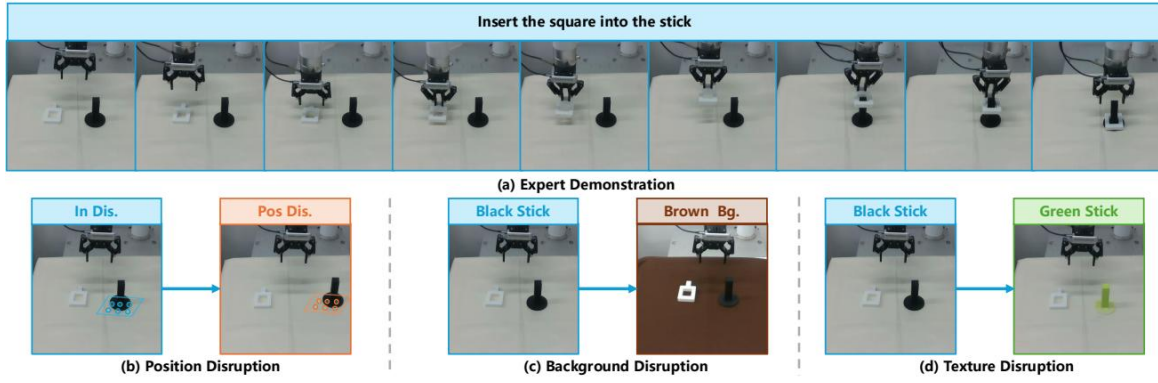


图 8: 带有扰动设置的真实世界实验示意。

为全面评估 APO, 我们不仅在分布内场景下开展实验, 还在三种不同的扰动设置下进行了测试, 如图 5(b-e) 所示:

1. 位置扰动 (Position Disruption): 改变 stick 的位置分布。
2. 背景扰动 (Background Disruption): 将桌布从白色更换为棕色。
3. 纹理扰动 (Texture Disruption): 将黑色 stick 替换为绿色 stick。

如表 5 所示, APO 在这些下游扰动场景中表现出较强的自适应能力。实验结果从经验上验证了该方法在非结构化环境中进行真实部署的实用价值。我们还将 APO 用于真实场景下的 π -FAST 模型微调。表 6 的结果表明, 相比其他方法, APO 能够稳定带来一致的性能提升。

5.8 4.7 失败场景下的纠错

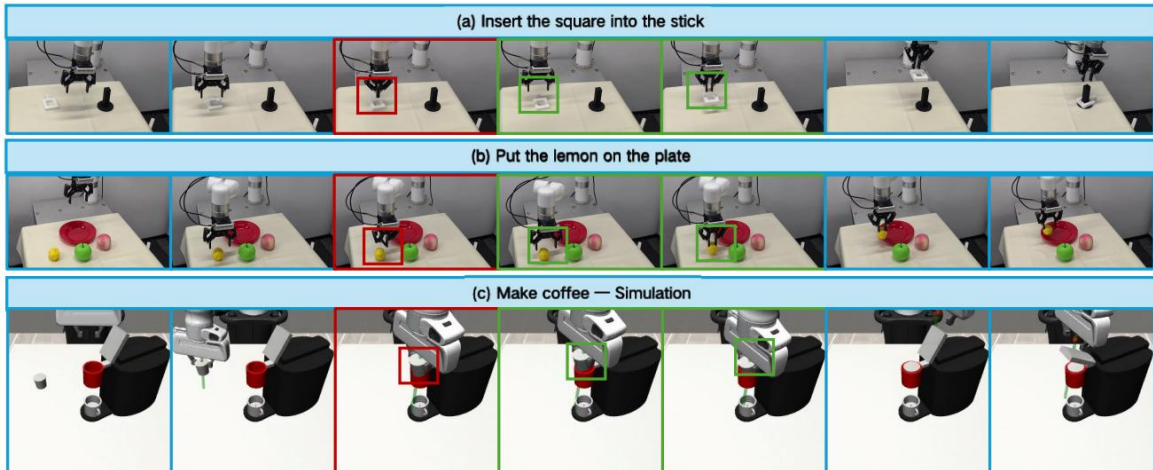


图 9: APO 的 rollout 轨迹。如红色和绿色粗框所示, APO 能够在失败场景中自主完成纠错。

本文提出的 APO 方法不仅能使模型规避失败模式, 还能在已经进入失败场景后进行自我纠正。如图 6 所示, 我们给出了 APO 在多个任务中的失败纠错示例, 展示了其学到的纠错策略。

具体而言, 在图 6(a,b) 中, 当模型初次抓取物体失败时, APO 能识别该失败并发起重新抓取尝试。类似地, 在图 6(c) 中, 当精确插入操作受阻时, APO 会学习迭代调整 gripper 的位置, 直到成功完成插入。这些示例表明, APO 已经成功学会从常见失败场景中恢复, 这也是其整体性能提升的直接原因。

6 5 结论

本文提出了 Action Preference Optimization (APO) 方法, 旨在在保持大规模 VLA 模型训练稳定性的同时, 充分挖掘失败轨迹中的有效信息。该方法建立在一个面向可靠部署的人机协作框架之上, 并利用带有 action-level 二值 desirability 信号的自适应重加权偏好优化算法, 对 VLA 模型进行稳定优化。借助 APO, 我们能够在 VLA 模型部署过程中持续推动模型性能改进。我们希望, APO 能为 VLA 模型在下游操纵任务上的高效、有效自适应提供有价值的启发。

讨论与未来工作 尽管本文研究了面向 VLA 模型的偏好对齐优化问题, 但实验目前仅基于自回归式 VLA 模型开展。未来工作还需要进一步考察更广泛的 VLA 框架, 包括基于回归的方法以及 diffusion policy 模型, 从而验证该方法在不同架构上的泛化能力。

7 参考文献

参考文献条目较多, 此处保留原论文英文文献列表, 请参阅原文 PDF 的 References 一节。

[附录因公式排版问题从译本中省略, 请参阅原文。]